Capstone-Projektentwurf: Ein differenzierbarer spektraler Physikmotor

Von gaußschen Wellenpaketen zum GPU-nativen, datengetriebenen Simulationskern

Faruk Alpay
Lightcap
AI Researcher
alpay@lightcap.ai

April 2026

Zusammenfassung

Diese Ausarbeitung formuliert nicht mehr nur ein Prüfungsproblem zur Quantenmechanik, sondern einen vollständigen Projektentwurf für ein Capstone-Vorhaben. Ausgangspunkt ist ein gaußsches Wellenpaket im unendlich tiefen Potentialtopf. Die zentrale Idee lautet: Aus demselben mathematischen Kern, der Superposition, Spektralzerlegung, Normerhaltung und zeitliche Entwicklung beschreibt, lässt sich ein moderner Physikmotor ableiten, wenn man die Zustandsrepräsentation, die numerische Diskretisierung und die Gradientenführung konsequent zusammen denkt.

Der vorgeschlagene Motor ist kein weiterer allgemeiner Rigid-Body-Solver. Stattdessen ist er als *differenzierbarer spektraler Physikmotor auf adaptiven gaußschen Paketen* konzipiert: ein Simulationskern, der physikalische Zustände sowohl als Basis-Koeffizienten als auch als räumliche Punkt-/Paketrepräsentationen verwaltet, auf GPU-Hardware effizient propagiert und aus Beobachtungsdaten zurück auf Zustände oder Materialparameter schließen kann. Gerade diese Verbindung aus Physik, Numerik, Datenrepräsentation und inverser Modellierung macht das Projekt technisch interessant und offen anschlussfähig an wissenschaftliches Rechnen, Computer Graphics, Robotik und datenintensive Workflows.

1 Die ehrliche Kurzantwort

Ja, aus dieser Mathematik kann ein sehr gutes Projekt entstehen — aber nicht automatisch ein allumfassender "Physikmotor für alles". Für einen wirklich starken Open-Source-Beitrag ist ein fokussierter Motor realistischer: nicht maximale Breite, sondern eine ungewöhnlich gute Kombination aus

1. klarer mathematischer Struktur,
2. starker numerischer Implementierbarkeit,
3. GPU-freundlicher Datenorganisation,
4. Differenzierbarkeit,
5. und visuell verständlichen Demos.

Die wahrscheinlich beste Richtung ist daher ein Motor, der *Wellen-, Feld- und Punktzustände* gemeinsam behandelt und sowohl *Vorwärtssimulation* als auch *inverse Rekonstruktion* beherrscht.

2 Projektidee

Arbeitstitel

Spectral Packet Engine (SPE)

Kernaussage

Der Motor repräsentiert einen physikalischen Zustand nicht nur auf einem Gitter, sondern in zwei komplementären Sichten:

- **spektral**: als Koeffizienten in einer Eigenbasis oder Fourier-/Sinusbasis,
- **räumlich**: als adaptive Sammlung lokalisierter gaußscher Pakete, die sich wie eine Punktwolke lesen lassen.

Damit entsteht eine Brücke zwischen Physik und Informatik: Für Physiker ist das eine Entwicklung in Eigenmoden; für Informatiker oder Data Engineers ist es gleichzeitig eine Datenpipeline aus Arrays, Punkten, Bins, sparsamen Strukturen und Reduktionen.

Warum gerade diese Idee?

Die Spektralform macht lineare Operatoren extrem sauber; die Paketform macht Visualisierung, Beobachtungsabgleich und lokale Verfeinerung einfach. Inverse Aufgaben — also *aus Messdaten zurück auf den Zustand oder auf Parameter zu schließen* — werden dadurch viel natürlicher als in einem klassischen, rein vorwärts orientierten Spiel-Physikmotor.

3 Was daran neu genug wäre, um Aufmerksamkeit zu erzeugen

Die Idee ist nicht, etwas zu behaupten, das es garantiert noch nie gab. Realistisch ist vielmehr: Ein Projekt bekommt dann viele Nutzer und Sterne, wenn es einen existierenden Bedarf deutlich besser trifft als bisher verfügbare Werkzeuge.

Für einen stark beachteten Motor wären insbesondere diese Eigenschaften entscheidend:

1. **GPU-first statt CPU-first.**
2. **Differenzierbar statt nur simulierbar.**
3. **Paket-/Punktdarstellung und Spektraldarstellung in einem gemeinsamen Kern.**
4. **Kurze Python-API plus nachvollziehbare mathematische Dokumentation.**
5. **Ein Killer-Demo: aus wenigen Beobachtungen Material- oder Zustandsparameter rekonstruieren.**

Der eigentliche "Wow-Effekt" läge also nicht in einem weiteren Integrator für starre Körper, sondern in einem Motor, der zugleich simuliert, visualisiert und schätzt.

4 Mathematischer Kern

4.1 Ausgangspunkt: Wellenpaket im Kasten

Wir beginnen mit dem bekannten Modell auf dem Intervall $0 < x < L$:

$$V(x) = \begin{cases} 0, & 0 < x < L, \\ \infty, & \text{sonst.} \end{cases}$$

Der Anfangszustand sei

$$\psi(x,0) = \begin{cases} A \exp\left(-\dfrac{(x-x_0)^2}{4\sigma^2}\right) \mathrm{e}^{\mathrm{i}k_0 x}, & 0 < x < L, \\ 0, & \text{sonst.} \end{cases}$$

mit

$$\hbar = 1, \qquad m = 1, \qquad L = 1, \qquad x_0 = 0.30, \qquad \sigma = 0.07, \qquad k_0 = 25.$$

Die Normierung folgt aus

$$1 = \int_0^L |\psi(x,0)|^2 \, \mathrm{d}x$$

und liefert

$$A^{-2} = \sqrt{\frac{\pi}{2}}\,\sigma \left[\operatorname{erf}\!\left(\frac{L - x_0}{\sqrt{2}\sigma}\right) + \operatorname{erf}\!\left(\frac{x_0}{\sqrt{2}\sigma}\right)\right].$$

Für die gewählten Parameter ergibt sich numerisch

$$A \approx 2.387305456.$$

4.2 Spektralzerlegung

Die Eigenfunktionen des Kastens sind

$$\phi_n(x) = \sqrt{\frac{2}{L}} \sin\!\left(\frac{n\pi x}{L}\right), \qquad E_n = \frac{n^2\pi^2\hbar^2}{2mL^2}.$$

Damit gilt

$$\psi(x,0) = \sum_{n=1}^{\infty} c_n \phi_n(x), \qquad c_n = \int_0^L \phi_n(x)\psi(x,0)\,\mathrm{d}x,$$

und die Zeitentwicklung lautet

$$\psi(x,t) = \sum_{n=1}^{\infty} c_n \phi_n(x) \mathrm{e}^{-\mathrm{i}E_n t/\hbar}.$$

4.3 Die Projektverallgemeinerung

Für einen Motor genügt nicht ein einzelnes Paket. Daher verallgemeinern wir die Zustandsdarstellung zu

$$\Psi(x,t) = \sum_{j=1}^{M} a_j(t) \exp\!\left(-\frac{(x-\mu_j(t))^2}{2s_j(t)^2}\right) \mathrm{e}^{\mathrm{i}k_j(t)x + \mathrm{i}\varphi_j(t)}.$$

Hier sind $a_j, \mu_j, s_j, k_j, \varphi_j$ dynamische Parameter lokaler Pakete. Parallel dazu kann derselbe Zustand projektiv in einer globalen Basis geschrieben werden:

$$\Psi(x,t) = \sum_{n=1}^{N} \hat{c}_n(t)\, \phi_n(x).$$

Genau diese Doppelrepräsentation ist der Kern des vorgeschlagenen Motors:

- global stabil im Spektralraum,
- lokal interpretierbar im Paketraum,
- differenzierbar gegenüber Parametern, Anfangsdaten und Beobachtungsfehlern.

5 Vom mathematischen Modell zum Motor

5.1 Forward Layer

Der Vorwärtskern berechnet die Zeitentwicklung eines Zustands unter einem gegebenen Operator $\mathcal{L}$, etwa

$$\partial_t \Psi = \mathcal{L}(\Psi; \theta),$$

wobei θ für Potentialparameter, Randbedingungen oder Materialparameter steht.

Für lineare Probleme ist eine Spektralmethode besonders attraktiv:

$$\hat{c}_n(t + \Delta t) = \hat{c}_n(t)\, \mathrm{e}^{-\mathrm{i}E_n \Delta t/\hbar}.$$

Für komplexere Modelle kommen Operator-Splitting, Projektion und lokale Korrekturschritte hinzu.

5.2 Observation Layer

Messungen erscheinen als diskrete Beobachtungen

$$y_i \approx \mathcal{H}[\Psi](\xi_i, t_i) + \varepsilon_i,$$

mit Beobachtungsoperator $\mathcal{H}$. Dies kann ein Punktwert, ein Integral über eine Zelle, ein Histogramm-Bin oder ein Bildpixel sein.

5.3 Inverse Layer

Die Differenzierbarkeit wird relevant, sobald man nicht nur simulieren, sondern auch schätzen will:

$$\mathcal{J}(\theta, \Psi_0) = \sum_i \left|\mathcal{H}[\Psi_{\theta,\Psi_0}](\xi_i, t_i) - y_i\right|^2 + \lambda\, \mathcal{R}(\theta, \Psi_0).$$

Der Motor ist dann erfolgreich, wenn er Gradienten nach θ und Ψ_0 robust und effizient liefern kann.

6 Warum das für Data Engineering anschlussfähig ist

Für einen Data Engineer ist der Clou nicht zuerst die Schrödinger-Gleichung, sondern die Struktur der Berechnung:

1. **State as table**: Jedes Paket ist ein Datensatz mit Feldern $(a_j, \mu_j, s_j, k_j, \varphi_j)$.
2. **State as columnar array**: Diese Felder liegen vektorisiert vor und sind damit GPU-freundlich.
3. **Binning und Aggregation**: Beobachtungen entstehen aus Histogrammen, Dichten, Integralen und Reduktionen.
4. **Sparse locality**: Nur lokal relevante Pakete beeinflussen lokale Beobachtungen stark.
5. **Streaming/updates**: Neue Sensorwerte können inkrementell in die Schätzung eingehen.
6. **Tail management**: Wie in großen Datensätzen muss man entscheiden, wann ein langer Spektral- oder Ereignisschwanz explizit geführt und wann er abgeschnitten wird.

Damit ist das Projekt nicht nur “Physik mit Code”, sondern ein echtes Architekturproblem zwischen numerischer Analysis und skalierbarer Datenverarbeitung.

7 Die Minimalversion, die realistisch in einem Capstone entsteht

Ein gutes Capstone-Projekt muss klein genug sein, um fertig zu werden, und stark genug, um zu beeindrucken. Die realistische erste Version ist:

1. **1D-Wellenmotor** auf einem endlichen Intervall,
2. **adaptive Gauß-Paket-Repräsentation**,
3. **spektrale Projektion** auf Sinusbasis,
4. **Vorwärtssimulation** mit Normkontrolle,
5. **inverse Rekonstruktion** von x_0, σ oder k_0 aus wenigen Beobachtungen,
6. **GPU-Backend** für Paketupdates und Beobachtungsoperatoren,
7. **visuelle Demos** als Notebook oder Web-App.

8 Numerischer Prototyp auf Basis des Kastenproblems

Für das Basismodell wurden $N = 200$ Moden verwendet. Die spektrale Trunkation liefert

$$\sum_{n=1}^{200} |c_n|^2 \approx 0.999999409,$$

also eine praktisch vollständige Erfassung der Norm. Die Bewegung des Pakets ist in Tabelle 1 zusammengefasst.

Tabelle 1: Zeitentwicklung des gaußschen Pakets im Kastenmodell.

t	$\langle x \rangle_t$	$\sigma_{\text{eff}}(t)$	$P_L(t)$	$P_R(t)$	$P_{\text{tot}}(t)$
0	0.300003	0.069993	0.997862	0.002137	0.999999
0.001	0.325004	0.070353	0.993559	0.006441	0.999999
0.003	0.375007	0.073192	0.956157	0.043842	0.999999
0.005	0.425009	0.078567	0.829933	0.170066	0.999999
0.01	0.550013	0.09999	0.308774	0.691226	0.999999

9 Die Verbindung von Abbildung 1 und Abbildung 2

Diese zwei Abbildungen sind didaktisch wichtiger als sie auf den ersten Blick wirken.

Erste Sicht: Punktwolke im Ortsraum. In Abbildung 1 sieht man zu jedem Zeitpunkt räumliche Stützstellen des Pakets. Ein datenorientierter Leser erkennt sofort eine Menge von Punkten, die man binned, streamt, verdichtet, filtert oder komprimiert.

Zweite Sicht: Tail im Modusraum. In Abbildung 2 sieht man, dass derselbe Zustand spektral nur auf relativ wenigen Moden stark sitzt, während der Rest als langer, aber dünner Tail ausläuft. Die relevante "Unendlichkeit" ist hier nicht $x \to \infty$, sondern der Grenzübergang $N \to \infty$ in der Basisauflösung.

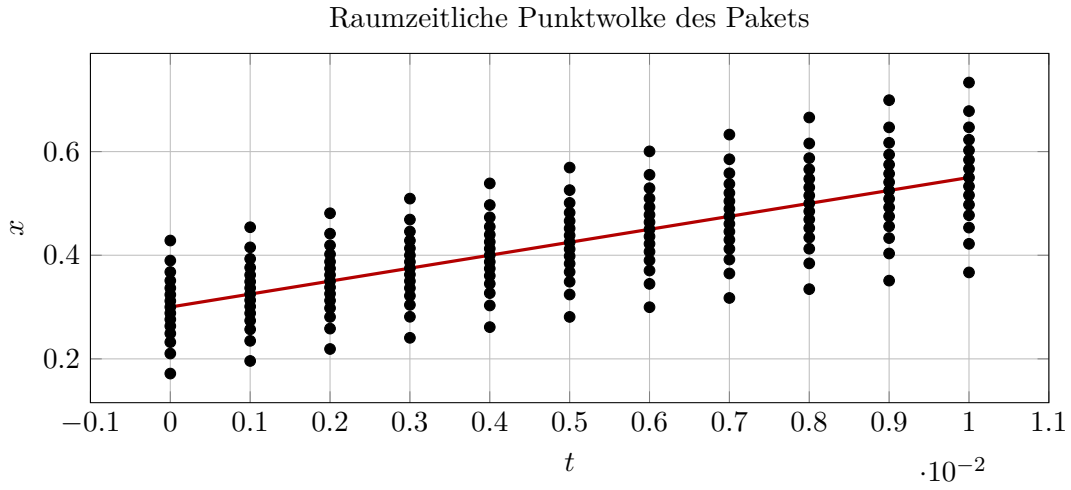

Abbildung 1: Die blauen Punkte zeigen diskrete Stützstellen der räumlichen Verteilung. Die rote Kurve markiert den Erwartungswert $\langle x \rangle_t$. Für die Engine-Interpretation ist wichtig: Der Zustand ist gleichzeitig als Punktwolke lesbar und als globaler Zustand mathematisch wohldefiniert.

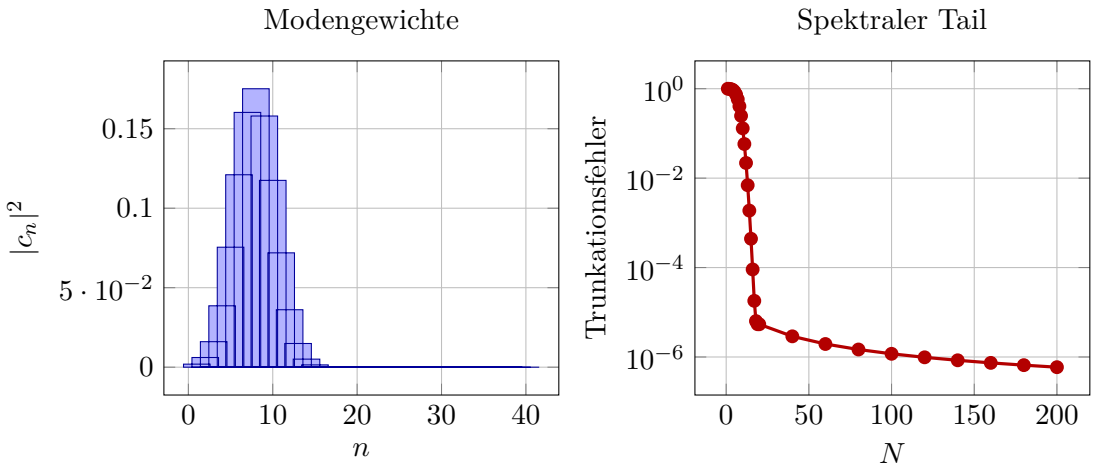

Abbildung 2: Links: dominierende Moden des Anfangszustands. Rechts: Der Fehler der Spektraltrunkation fällt mit wachsendem N. Für einen Motor ist genau das die operative Frage: Wo endet die "wichtige Masse" des Zustands, und wie verwaltet man den langen Tail effizient?

Warum das projektentscheidend ist. Ein guter Motor muss beides können:

- im Ortsraum lokal, interaktiv und datenfreundlich bleiben,
- im Modusraum global stabil und numerisch kontrollierbar bleiben.

Genau diese Dualität macht das Projekt stärker als einen klassischen Solver, der nur ein einziges Zustandsformat kennt.

10 Architekturvorschlag

10.1 Modul 1: State Representation

- `PacketState`: Arrays für $a_j, \mu_j, s_j, k_j, \varphi_j$.
- `SpectralState`: Arrays für $\hat{c}_n$.
- `Projector`: Vorwärts- und Rückprojektion zwischen beiden Darstellungen.

10.2 Modul 2: Time Stepping

- exakte spektrale Propagation für lineare Teiloperatoren,
- Splitting für Potential und Drift,
- adaptive Reprojektion, wenn Pakete zu breit, zu schmal oder zu zahlreich werden.

10.3 Modul 3: Observation and Loss

- Punktbeobachtungen,
- Histogramme/Bins,
- Bild- oder Heatmap-Render,
- zusammengesetzte Verluste für Zustandsschätzung.

10.4 Modul 4: GPU Runtime

- vektorisierte Paketupdates,
- parallele Reduktionen für Normen, Momente und Verluste,
- sparse Nachbarschaftsstrukturen für lokale Wechselwirkungen,
- Debug-Modus mit CPU-Referenz.

11 Was ein starkes GitHub-Projekt daraus macht

Ein Projekt dieser Art hätte dann gute Chancen auf starke Resonanz, wenn es nicht nur eine gute Idee, sondern auch ein gutes *Produkt* ist. Praktisch bedeutet das:

1. **Drei exzellente Demos statt zehn halbfertige Beispiele.**
2. **Installierbar mit einem Befehl.**
3. **Jede Demo zeigt Physik, Numerik und Rekonstruktion zugleich.**
4. **Jede Kernformel steht im Repo genauso klar wie der Code.**
5. **Tests prüfen Normerhaltung, Energiefehler und Gradientenkorrektheit.**

Wenn der Motor zusätzlich von Video-, Bild- oder Sensorbeobachtungen zurück auf Zustände schließen kann, ist er nicht nur für Physikstudenten, sondern auch für Robotik-, Graphics- und ML-Leute interessant.

12 Arbeitsplan für ein Semester

Woche	Ziel
1–2	Mathematisches Minimalmodell, Referenzlösung im Kasten, Tests für Norm und Spektralprojektion
3–4	Paketrepräsentation und Projektion zwischen Paket- und Spektralraum
5–6	CPU-Prototyp für Vorwärtssimulation und Visualisierung
7–8	GPU-Portierung der Paketupdates und Beobachtungsoperatoren
9–10	Inverses Problem: Schätzung von x_0, σ, k_0 aus wenigen Beobachtungen
11–12	Dokumentation, Demo-Notebooks, Benchmark, Fehleranalyse

13 Bewertungskriterien

Das Projekt ist dann erfolgreich, wenn am Ende nicht nur "es läuft" gesagt werden kann, sondern konkrete Kriterien erfüllt sind:

1. **Korrektheit**: Normerhaltung, plausible Konvergenz des Tails, reproduzierbare Referenzfälle.
2. **Effizienz**: spürbarer Vorteil der GPU-Version bei vielen Paketen oder vielen Beobachtungen.
3. **Interpretierbarkeit**: klare Verbindung zwischen Formel, Datenstruktur und Abbildung.
4. **Differenzierbarkeit**: Gradienten stimmen mit Finite-Differences-Checks überein.
5. **Nützlichkeit**: inverse Rekonstruktion funktioniert in mindestens einem überzeugenden Demo-Szenario.

14 Schlussfolgerung

Aus der Mathematik des gaußschen Wellenpakets allein entsteht noch kein universeller Physikmotor. Wohl aber entsteht daraus ein sehr starkes Capstone-Projekt mit überraschend moderner Stoßrichtung: ein differenzierbarer, GPU-nativer Simulationskern, der globale Spektralmethoden mit lokaler Punkt-/Paketdarstellung verbindet.

Genau darin liegt die eigentliche Stärke der Idee. Die Formel

$$\int_0^L |\psi(x,t)|^2 \, \mathrm{d}x = 1$$

steht am Anfang als Aussage über Existenz, Norm und Verteilung eines Zustands. Im Projekt wird daraus eine Architekturfrage: Wie speichert, propagiert, beobachtet und optimiert man einen Zustand, der lokal wie eine Punktwolke aussieht, global aber durch wenige dominante Moden organisiert wird?

Wenn diese Frage sauber beantwortet wird, ist das Ergebnis nicht nur ein guter Bericht, sondern ein Repo mit echter Außenwirkung.

Literatur

[1] B. Garbrecht, *Theoretische Physik 3 – Quantenmechanik*, Technische Universität München, Vorlesungsskript.
https://www.ph.nat.tum.de/fileadmin/w00bya/t70/QM/Theoretische_Physik_3_-_Quantenmechanik_-_Skript.pdf

[2] T. Brandes, *Quantenmechanik*, Institut für Theoretische Physik, TU Berlin.
https://www1.itp.tu-berlin.de/brandes/qm2009.pdf

[3] N. Thuerey et al., *Real Time Physics – Class Notes*.
https://ge.in.tum.de/download/thuerey/RealTimePhysics_course.pdf

[4] *HPC – Algorithms and Applications*, Technische Universität München.
https://www5.in.tum.de/wiki/index.php/HPC_-_Algorithms_and_Applications_-_Winter_15

[5] T. Neumann, *Foundations of Data Engineering*, Technische Universität München.
https://db.in.tum.de/teaching/ws1718/foundationsde/chapter1.pdf?lang=de

[6] *Physique numérique (pour SPH)*, EPFL, fiche de cours.
https://edu.epfl.ch/coursebook/fr/physique-numerique-pour-sph-PHYS-210

[7] T. Roscilde, B. Guiselin, *UE Physique Numérique – Computational Physics (M1)*, ENS de Lyon.
https://perso.ens-lyon.fr/tommaso.roscilde/CompPhys-M1/Slide_PhysiqueNumerique_M1.pdf

[8] T. Roscilde, *Tutorat de mécanique quantique*, ENS de Lyon.
https://perso.ens-lyon.fr/tommaso.roscilde/Fascicule2010-2011.pdf

[9] *NVIDIA Warp Documentation.*
https://nvidia.github.io/warp/

[10] *Differentiability – Warp User Guide.*
https://nvidia.github.io/warp/user_guide/differentiability.html

[11] *Build Accelerated, Differentiable Computational Physics Code for AI with NVIDIA Warp*, NVIDIA Technical Blog.
https://developer.nvidia.com/blog/build-accelerated-differentiable-computational-physics-code-for-ai-with-nvidia-warp/

[12] *Taichi Lang.*
https://github.com/taichi-dev/taichi

[13] *Differentiable Programming – Taichi Docs.*
https://docs.taichi-lang.org/docs/differentiable_programming

[14] *Genesis – A generative world for robotics and embodied AI.*
https://github.com/Genesis-Embodied-AI/Genesis